

down or starts up less frequently, then this is a good choice. To launch Anacron via Cron, create a Cron table entry in the `/etc/crontab` file:

```
@hourly * * * * * root anacron
```

For the job above, Cron executes Anacron once every hour, and Anacron executes jobs only if necessary.

Running Anacron via a non-root user: In our earlier discussion, we have seen how the root user can run Anacron. But it is a very common requirement to execute Anacron via an ordinary or non-root user. We can achieve this by simply creating a private Anacron table and by specifying a different spool area. To run Anacron via an ordinary user add the following entry into the user's Cron table or execute Anacron via login scripts:

```
crontab -t /home/jerry/jerry.anacrontab -S /home/jerry/
crontab_spool
```

...where the `-t` option is used to specify a private Anacron table and the `-S` option is used to specify the spool area that stores the timestamps file.

Insights about Crontab

Anacron is sensitive to signals. The signal `'SIGUSR1'` can be used to stop Anacron gracefully. After receiving the

`'SIGUSR1'` signal, if there are any running jobs, Anacron waits to finish them and exits gracefully. To know more about signals, refer to the manual page of the `'kill'` command from your distribution.

By default, Anacron forks out and creates child processes. The parent process exits immediately and child processes start their execution in the background. By providing the `'-d'` option, we can instruct Anacron not to fork out in the background. In this mode, Anacron will print all messages to the standard error stream as well as send them to the `'syslog'` daemon. Additionally, it will also e-mail the output according to the `MAILTO` setting.

Before job execution, Anacron does some pre-checks. Reading the timestamps file is one of them. Anacron then decides whether job execution is needed or not. We can override this default behaviour by using the `'-f'` option, which forces Anacron to execute the job by ignoring timestamps.

We can also perform dry runs of jobs for testing purposes. The `'-u'` option of Anacron just updates the timestamps to the current date, but in reality does not run any job.

We can instruct Anacron to start job execution in serial order. The `'-s'` option of Anacron takes care of that. In serial job execution, the next job will be started only when the previous job completes its execution.

Anacron uses the `/var/spool/anacron` directory as a spool area to store the timestamps file. We can also use other directories as the spool area by specifying an argument to the `'-S'` option. This is needed when a non-root user wants to execute a job through Anacron.

Caveats

Although Anacron is a great utility and works intelligently by taking care of missed jobs, it also has some shortcomings. Let us discuss a few of them.

- For Anacron, the smallest possible granularity is days. It does not deal with hours or minutes.
- Anacron creates the timestamps file per job in the spool area, and those files never get removed automatically. If the user removes a particular job entry from an Anacron table, then he has to remove the timestamps file manually.
- For each job execution, Anacron uses up to two file descriptors; so we can easily run out of descriptors if we run a large number of jobs. Please note that this limit varies from one version of GNU/Linux to another.

We have seen that, by combining Anacron with other utilities, we can manage a GNU/Linux system more efficiently. These simple but powerful command-line utilities make GNU/Linux more interesting and reliable. 

By: Narendra Kangarkar

The author is a FOSS enthusiast and loves exploring anything related to open source. He can be reached at narendrakangarkar@gmail.com



2 Simple Ways To Stay Up-to-date With India's Electronics Industry

Want Latest News?
Check out EfyTimes.com

Want Industry Trends?
Check out Electronics2b.com

For any queries, please contact our team at efyenq@efy.in OR +91-11-26810601